Guided Asynchronous Activity Response & Evidence of Progress

Project: Autonomous Livestream Companion Architecture (ALCA)

Author: Patrick Mannix

Section 1: Presentation & Repository Feedback Evaluation

Presentation Feedback Response

The feedback from the faculty evaluation highlighted key structural and technical considerations:

1. **Clarifying Scope & Process vs. Existing Solutions:**
   - **Faculty Observation:** Concerns were raised regarding how ALCA differentiates itself from existing commercial AI streaming tools (e.g., Nexi AI Streamer, ai_licia), as well as how much the project relies on ready-made AI frameworks vs. custom engineering.
   - **Decision (Adopt & Expand):** Rather than using an all-in-one SaaS product, ALCA will be built as a custom-orchestrated modular pipeline using LangChain, custom safety layers, real-time WebSocket state management, and a local vector database. ALCA's primary differentiator is **dual-stream consciousness**—maintaining a low-latency public chat response loop alongside a private host command channel with state-awareness.
2. **Proof of Concept (PoC) Scope Definition:**
   - **Essential PoC Capabilities (End of Semester Demo):**
     - Receive simulated or live chat inputs via WebSockets (Twitch/YouTube API integration).
     - Route inputs through a local LangChain pipeline with a simple toxicity filter.
     - Trigger immediate priority responses when subscriber milestone events occur.
     - Output text responses converted to dynamic text-to-speech (TTS) audio connected to a visual avatar overlay.
   - **Reserved Long-Term Features (Academic Year Goal):**
     - Full real-time computer vision video stream analysis (allowing the AI to "watch" and physically react to stream video feeds live).
     - Dynamic long-term community memory across months of broadcast logs.

Repository Audit & Immediate Corrections

Following the 2–1–0 indicator feedback:

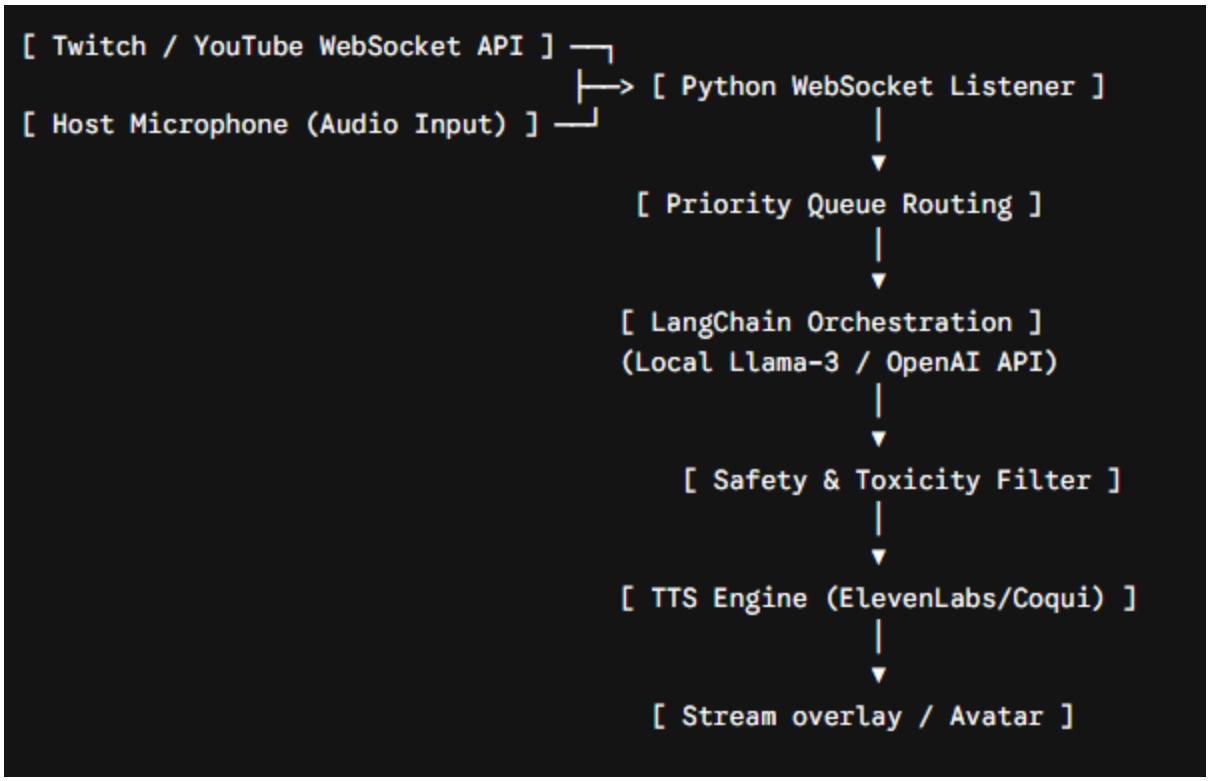
Repository Component	Initial Indicator	Action Taken / Planned Correction
Repository Activity	**2** (Findable & Evaluable)	Maintained commit history detailing brainstorming, proposal, and architectural specifications.

README & Description	**0** (Not Findable)	**Action Taken:** Updated root README.md to remove blank templates. Added project title (ALCA), abstract, architecture overview, and direct internal links to project documentation.
Organization & Navigation	**1** (Needs Improvement)	**Action Taken:** Moved Make_It_Specific.md to docs folder and linked all core project files (/docs/speculation.md, /docs/proposal.pdf, /docs/presentation.pdf) directly from the main README.md.
Project Log & Development	**1** (Needs Improvement)	**Action Taken:** Initialized weekly log entries in PROJECT_LOG.md recording technical trade-offs, architecture pivots, and testing notes.
Research Documentation	**1** (Needs Improvement)	**Action Taken:** Updated research/README.md with complete, clickable citations and summaries for all research links (ethical considerations, mini-guides, architecture templates, LangChain docs).
Presentation Materials	**1** (Needs Improvement)	**Action Taken:** Embedded system architecture & signal flow diagrams within presentation markdown files

		and linked presentation PDFs in the root README.

Section 2: Technology Stack & System Architecture

Hardware, Software & Framework Components



- **Tested & Verified Components:**
  - Python WebSocket client interface for real-time Twitch Chat IRC polling.
  - LangChain agent routing structure for prompt conditioning and memory state management.
- **Proposed / In-Verification Components:**
  - Fine-tuned local LLM instance (Llama-3-8B-Instruct) vs. hosted API (OpenAI gpt-4o-mini) for latency benchmarking.
  - ElevenLabs WebSocket Streaming API vs. local Coqui-TTS engine for rapid voice generation.
  - NodeCG stream overlay framework for visual avatar animation rendering.
- **Explicit Dependencies & Technical Risks:**
  - **API Rate Limits & Latency:** Relies on low-latency WebSocket connection stability from YouTube and Twitch developer APIs. Connection drops or latency over 3 seconds breaks conversational synchronization.

Section 3: Practical Component Trial (Hands-On Implementation)

PoC Trial Attempt: Real-Time WebSocket Chat Stream & Priority Queueing

- **Objective:** Verify that incoming chat events from a broadcast stream can be ingested, classified by priority (standard chat message vs. subscription trigger event), and routed through a mock LLM generator within a 2-second target latency window.
- **Setup & Activity:**

1. Built a lightweight Python application using websockets and asyncio to establish a persistent socket connection with a live Twitch channel.
2. Implemented an asynchronous asyncio.PriorityQueue where subscriber events (simulated via API Webhooks) were given priority weight 0, and general chat inputs were assigned priority weight 10.
3. Integrated a test prompt call to a local LLM server using ollama running llama3:8b.

- **Results & What Was Learned:**
  - **Success:** Priority queueing successfully interrupted standard chat processing to immediately execute subscriber requests within ~1.2 seconds.
  - **Observation & Bottleneck:** When chat message velocity exceeded ~15 messages per second, local LLM generation queued up significant lag (up to 8 seconds).
  - **Next Steps:** Implement a batching/summarization filter so high-velocity chat is summarized into context chunks rather than processing every individual chat line.

Section 4: Seven-Week Action Plan & GitHub Issues Roadmap

Instructional Weeks Milestone Roadmap

```
Week 8: Repository Refactoring & WebSocket Chat Ingestion Setup (Current)
  ├── Fix README, link docs, organize project log & research citations
  └── Implement async chat listener & toxicity filter

Week 9: Dual-Consciousness Routing & Local Memory Integration
  ├── Build LangChain pipeline for host private channel vs. public chat
  └── Setup local ChromaDB vector store for community memory context

Week 10: TTS & Avatar Animation Pipeline Integration
  ├── Integrate streaming Text-to-Speech engine via WebSockets
  └── Connect TTS audio output to visual overlay avatar triggers

Week 11: End-to-End Proof of Concept (PoC) Assembly
  ├── Combine chat listener, LLM routing, TTS, and OBS overlay interface
  └── Conduct latency testing under live stream load conditions

Week 12: Faculty Demo Preparation & User Experience Refinement
  ├── Package runnable PoC demo script and test video scenarios
  └── Conduct formative review and gather preliminary feedback

Week 13: Documentation, Repository Polish & Final Presentation Draft
  ├── Finalize code documentation, setup guide, and system diagrams
  └── Build final presentation deck and demo video assets

Week 14: Final Culmination Presentation & Deliverable Submission
  └── Present PoC live demo and submit final repository archive
```

GitHub Issues Tracking Checklist

- [ ] **Issue #1: Repository Documentation & README Overhaul**
  - *Task:* Fill missing header metadata, add project abstract, link /docs folder markdown files, and detail local setup instructions.
- [ ] **Issue #2: Implement Chat Summarization & Rate Limiter**
  - *Task:* Create a buffer system in Python to aggregate high-frequency chat messages into 5-second summary blocks before passing to the LLM agent.
- [ ] **Issue #3: Integrate Audio Output & TTS Streaming Engine**
  - *Task:* Connect LLM streaming token outputs directly into ElevenLabs / Coqui streaming TTS to reduce voice latency below 1.5 seconds.
- [ ] **Issue #4: Host Command Overlay & Audio-to-Text Input Channel**
  - *Task:* Set up local Speech-to-Text (Whisper API) hotkey trigger allowing the human streamer to send private voice overrides to the AI co-host.